



Tecnológico Nacional de México

Instituto Tecnológico de Chiná

Nombre de la carrera:

Ingeniería en Informática.

Nombre de la asignatura:

Programación cliente - servidor.

Semestre y grupo:

7AI

Actividad:

Registro de práctica #4 Implementación de un corte vertical mínimo.

Nombre del alumno:

Aaron Jesús Eloy Aguilar León.

Chi Chablé Miguel Eduardo.

Ek Noh Edgar Ernesto.

Nombre del docente:

Jenner Noe Che Méndez.

Lugar y fecha:

San Francisco de Campeche, Campeche a 03 de Octubre de 2026.

Índice

2.- Objetivo.	3
3.- Descripción de la funcionalidad.	3
4.- Arquitectura utilizada.....	3
5.- Diagrama del corte vertical.....	3
6.- Contrato API.	4
7.- Request Json.....	5
8.- Response Json.	5
9.- Código de cliente.	6
10.- Código del servicio.....	6
11.- Estructura de la base de datos.	7
12.- Evidencia de ejecución.	7
13.- Captura de Network.	8
14.- Evidencia de 201 Created.	9
15.- Evidencia de error.	10
16.- Explicación de la regla de negocio.	12
17.- Conclusiones.	12

2.- Objetivo.

Desarrollar e implementar un corte vertical mínimo (Vertical Slice) para el módulo de reservación de laboratorios, conectando la interfaz de usuario en el cliente, la API REST en el servidor, las reglas de validación y la persistencia en una base de datos SQLite, garantizando el manejo adecuado de los códigos de respuesta HTTP (201, 422 y 409) mediante el uso del equipo local.

3.- Descripción de la funcionalidad.

Permite a los usuarios registrar una solicitud de reservación de laboratorio especificando el nombre del laboratorio, fecha, hora de inicio, hora de fin y motivo.

Entonces, el sistema valida la integridad de los datos recibidos, aplica la regla de negocio para evitar cruces o empalmes de horarios en un mismo laboratorio y almacena las reservaciones en la base de datos, refrescando la lista en tiempo real en el cliente web.

4.- Arquitectura utilizada.

Se utilizó una arquitectura cliente-servidor descentralizada en capas (Corte Vertical):

- 1.- Cliente (Frontend): Interfaz Web construida con HTML5, CSS3 y JavaScript nativo (fetch API).
- 2.- Servidor (Backend): API REST desarrollada con FastAPI a través de (Python) ejecutada sobre el servidor ASGI Uvicorn.
- 3.- Base de datos: SQLite relacional para la persistencia local de las reservaciones.

5.- Diagrama del corte vertical.

[Cliente: HTML/JS]

|

| 1.- Request POST /api/reservaciones (JSON)



[Servidor: FastAPI]

| 2.- Validación de esquema Pydantic (Tipo de datos y Hora Inicio < Hora Fin)

| 3.- Verificación de regla de negocio (Conflicto de horario)



[Persistencia: SQLite]

| 4.- INSERT INTO reservaciones

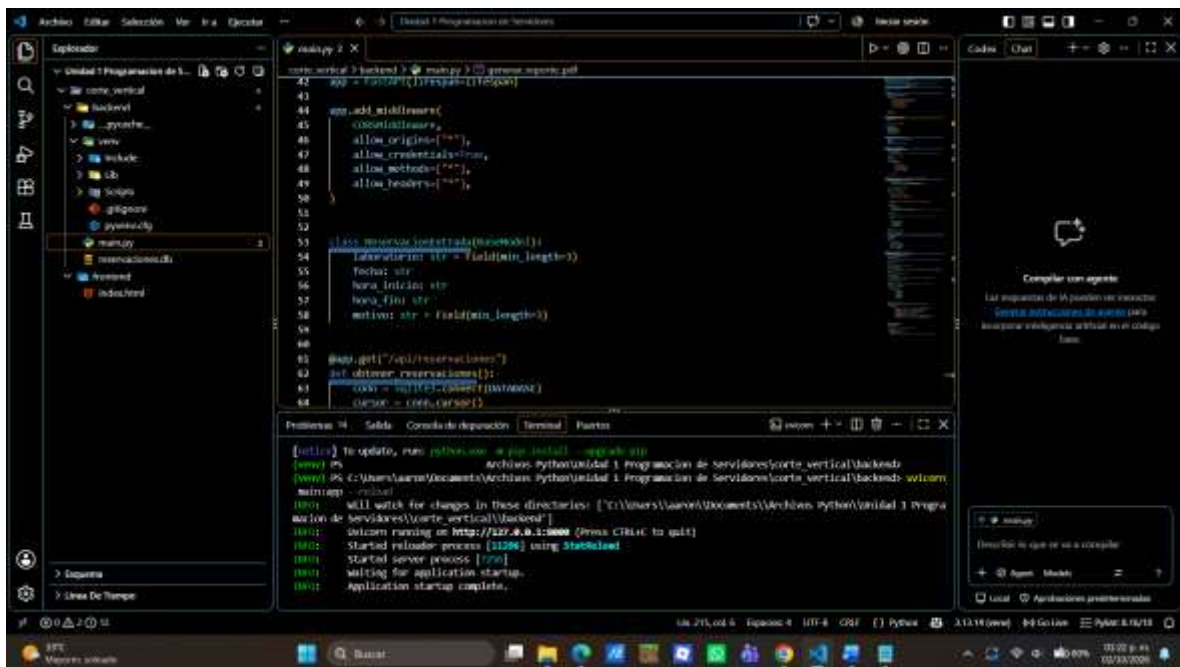


[Cliente: HTML/JS] ← 5.- Response JSON + Código HTTP (201 / 422 / 409)

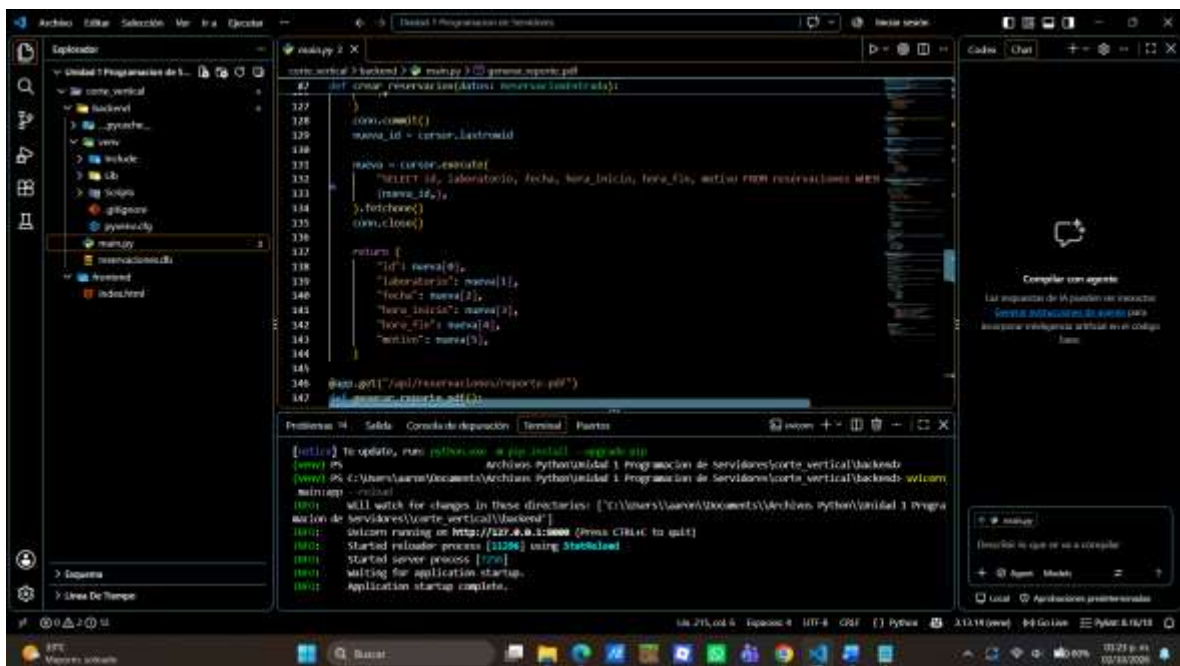
6.- Contrato API.

Endpoint	Método	Descripción	Código HTTP Esperado
/api/reservaciones	GET.	Obtiene la lista de reservaciones de forma completa.	200 OK.
/api/reservaciones	GET.	Se registra una nueva reservación de laboratorio.	201 Created. 422 Unprocessable Entity. 409 Conflict.

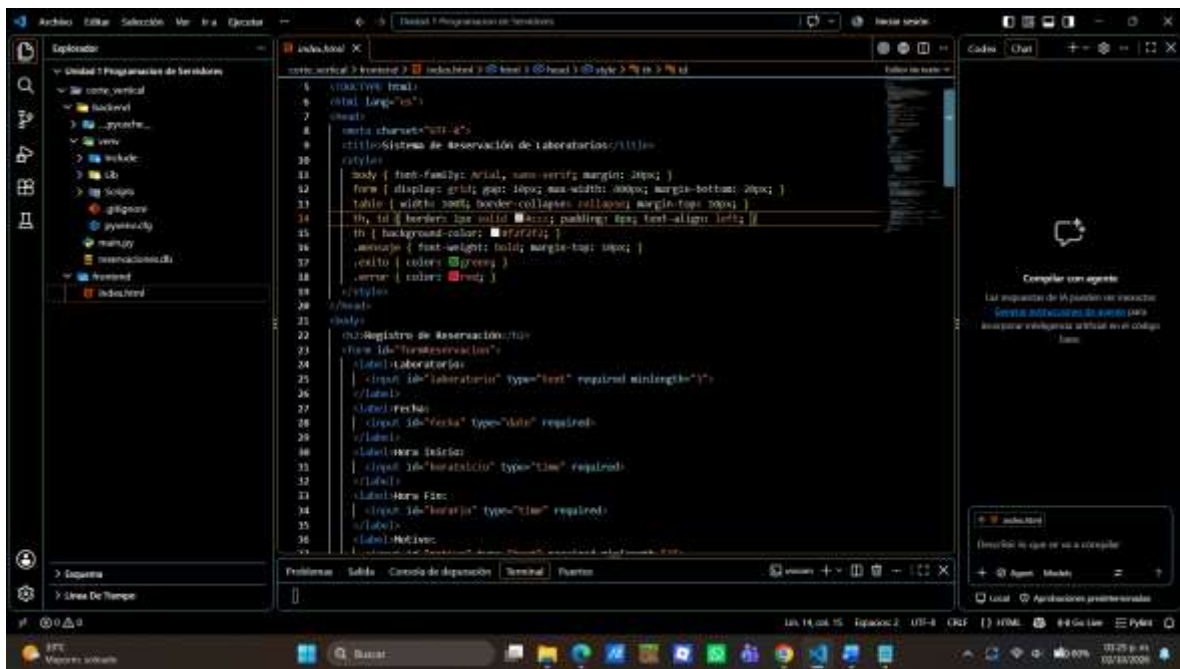
7.- Request Json.



8.- Response Json.



9.- Código de cliente.

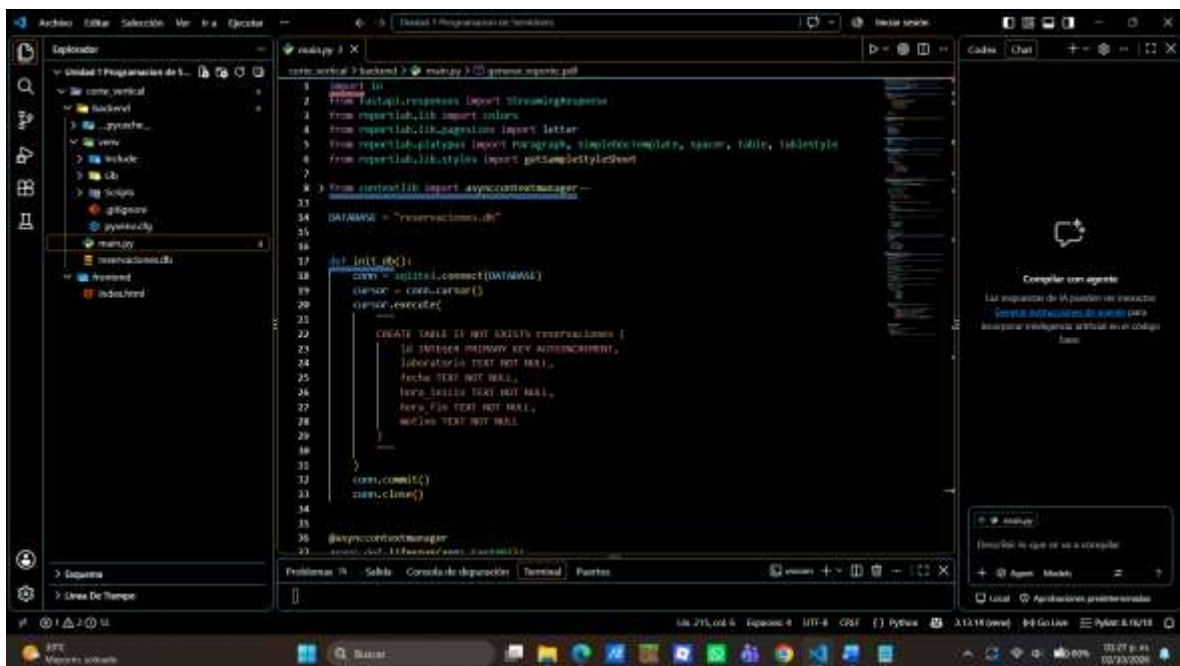


```

1 <DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Sistema de Reservación de laboratorios</title>
6   <style>
7     body { font-family: Arial, sans-serif; margin: 20px; }
8     form { display: grid; gap: 10px; max-width: 600px; margin-bottom: 20px; }
9     table { width: 100%; border-collapse: collapse; margin-top: 10px; }
10    tr { background-color: #e0f2f1; padding: 5px; text-align: left; }
11    th { background-color: #e0f2f1; padding: 5px; }
12    td { padding: 5px; }
13    input { width: 100%; height: 30px; border: 1px solid #ccc; }
14    input[type="text"] { border: 1px solid #ccc; }
15    input[type="date"] { border: 1px solid #ccc; }
16    input[type="time"] { border: 1px solid #ccc; }
17    input[type="text"] { border: 1px solid #ccc; }
18    input[type="submit"] { background-color: #00bcd4; color: white; padding: 10px 20px; border: none; }
19  </style>
20 </head>
21 <body>
22   <h2>Registro de Reservación</h2>
23   <div id="formReservacion">
24     <div id="laboratorio">
25       <input id="laboratorio" type="text" required minlength="3">
26     </div>
27     <div id="fecha">
28       <input id="fecha" type="date" required>
29     </div>
30     <div id="hora">
31       <input id="hora" type="time" required>
32     </div>
33     <div id="horaFin">
34       <input id="horaFin" type="time" required>
35     </div>
36     <div id="nombre">
37       <input id="nombre" type="text" required minlength="3">
38     </div>
39     <input type="submit" value="Reservar">
40   </div>
41 </body>
42 </html>

```

10.- Código del servicio.



```

1 from flask import Flask, request, jsonify
2 from flask_cors import CORS
3 from flask_restful import Resource, Api
4 from flask_sqlalchemy import SQLAlchemy
5 from flask_migrate import Migrate
6 from flask_login import LoginManager
7 from flask_bcrypt import Bcrypt
8 from flask_mail import Mail
9 from flask_socketio import SocketIO
10 from flask_sse import SSEManager
11 from flask_jwt_extended import JWTManager
12 from flask_jwt_refresh import RefreshToken
13 from flask_jwt_refresh import RefreshToken
14 from flask_jwt_refresh import RefreshToken
15 from flask_jwt_refresh import RefreshToken
16 from flask_jwt_refresh import RefreshToken
17 from flask_jwt_refresh import RefreshToken
18 from flask_jwt_refresh import RefreshToken
19 from flask_jwt_refresh import RefreshToken
20 from flask_jwt_refresh import RefreshToken
21 from flask_jwt_refresh import RefreshToken
22 from flask_jwt_refresh import RefreshToken
23 from flask_jwt_refresh import RefreshToken
24 from flask_jwt_refresh import RefreshToken
25 from flask_jwt_refresh import RefreshToken
26 from flask_jwt_refresh import RefreshToken
27 from flask_jwt_refresh import RefreshToken
28 from flask_jwt_refresh import RefreshToken
29 from flask_jwt_refresh import RefreshToken
30 from flask_jwt_refresh import RefreshToken
31 from flask_jwt_refresh import RefreshToken
32 from flask_jwt_refresh import RefreshToken
33 from flask_jwt_refresh import RefreshToken
34 from flask_jwt_refresh import RefreshToken
35 from flask_jwt_refresh import RefreshToken
36 from flask_jwt_refresh import RefreshToken
37 from flask_jwt_refresh import RefreshToken
38 from flask_jwt_refresh import RefreshToken
39 from flask_jwt_refresh import RefreshToken
40 from flask_jwt_refresh import RefreshToken
41 from flask_jwt_refresh import RefreshToken
42 from flask_jwt_refresh import RefreshToken
43 from flask_jwt_refresh import RefreshToken
44 from flask_jwt_refresh import RefreshToken
45 from flask_jwt_refresh import RefreshToken
46 from flask_jwt_refresh import RefreshToken
47 from flask_jwt_refresh import RefreshToken
48 from flask_jwt_refresh import RefreshToken
49 from flask_jwt_refresh import RefreshToken
50 from flask_jwt_refresh import RefreshToken
51 from flask_jwt_refresh import RefreshToken
52 from flask_jwt_refresh import RefreshToken
53 from flask_jwt_refresh import RefreshToken
54 from flask_jwt_refresh import RefreshToken
55 from flask_jwt_refresh import RefreshToken
56 from flask_jwt_refresh import RefreshToken
57 from flask_jwt_refresh import RefreshToken
58 from flask_jwt_refresh import RefreshToken
59 from flask_jwt_refresh import RefreshToken
60 from flask_jwt_refresh import RefreshToken
61 from flask_jwt_refresh import RefreshToken
62 from flask_jwt_refresh import RefreshToken
63 from flask_jwt_refresh import RefreshToken
64 from flask_jwt_refresh import RefreshToken
65 from flask_jwt_refresh import RefreshToken
66 from flask_jwt_refresh import RefreshToken
67 from flask_jwt_refresh import RefreshToken
68 from flask_jwt_refresh import RefreshToken
69 from flask_jwt_refresh import RefreshToken
70 from flask_jwt_refresh import RefreshToken
71 from flask_jwt_refresh import RefreshToken
72 from flask_jwt_refresh import RefreshToken
73 from flask_jwt_refresh import RefreshToken
74 from flask_jwt_refresh import RefreshToken
75 from flask_jwt_refresh import RefreshToken
76 from flask_jwt_refresh import RefreshToken
77 from flask_jwt_refresh import RefreshToken
78 from flask_jwt_refresh import RefreshToken
79 from flask_jwt_refresh import RefreshToken
80 from flask_jwt_refresh import RefreshToken
81 from flask_jwt_refresh import RefreshToken
82 from flask_jwt_refresh import RefreshToken
83 from flask_jwt_refresh import RefreshToken
84 from flask_jwt_refresh import RefreshToken
85 from flask_jwt_refresh import RefreshToken
86 from flask_jwt_refresh import RefreshToken
87 from flask_jwt_refresh import RefreshToken
88 from flask_jwt_refresh import RefreshToken
89 from flask_jwt_refresh import RefreshToken
90 from flask_jwt_refresh import RefreshToken
91 from flask_jwt_refresh import RefreshToken
92 from flask_jwt_refresh import RefreshToken
93 from flask_jwt_refresh import RefreshToken
94 from flask_jwt_refresh import RefreshToken
95 from flask_jwt_refresh import RefreshToken
96 from flask_jwt_refresh import RefreshToken
97 from flask_jwt_refresh import RefreshToken
98 from flask_jwt_refresh import RefreshToken
99 from flask_jwt_refresh import RefreshToken
100 from flask_jwt_refresh import RefreshToken

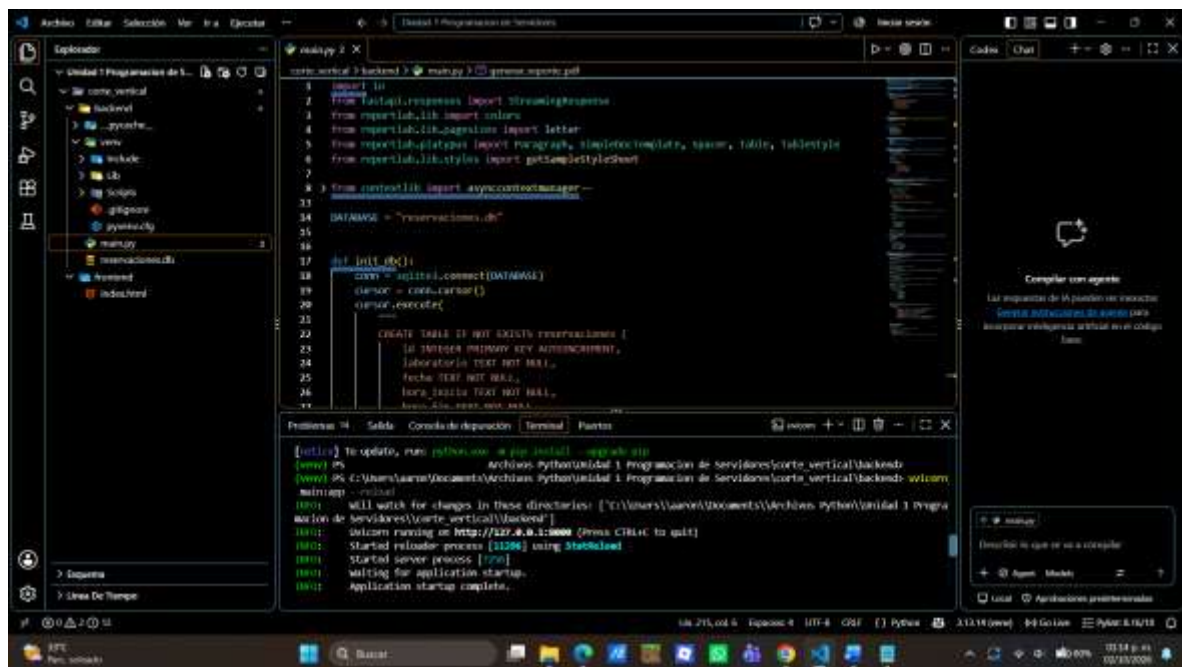
```

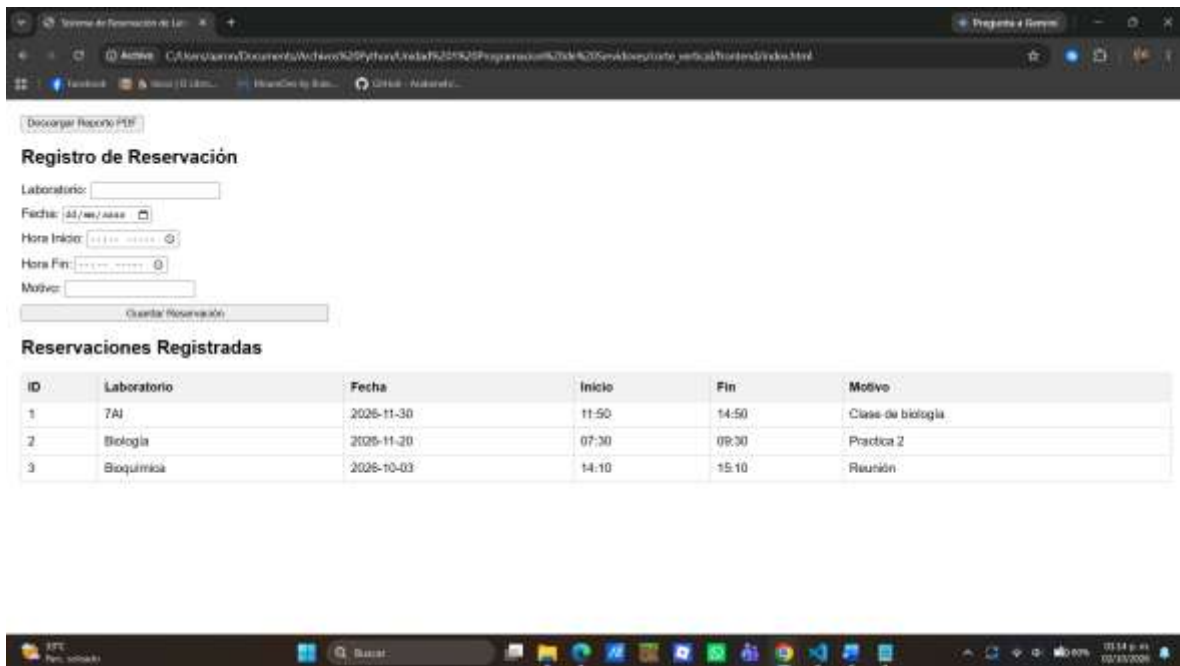
11.- Estructura de la base de datos.

- Tabla: reservaciones
- Motor: SQLite (reservaciones.db)
- Esquema SQL:

```
CREATE TABLE IF NOT EXISTS reservaciones (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    laboratorio TEXT NOT NULL,  
    fecha TEXT NOT NULL,  
    hora_inicio TEXT NOT NULL,  
    hora_fin TEXT NOT NULL,  
    motivo TEXT NOT NULL  
);
```

12.- Evidencia de ejecución.



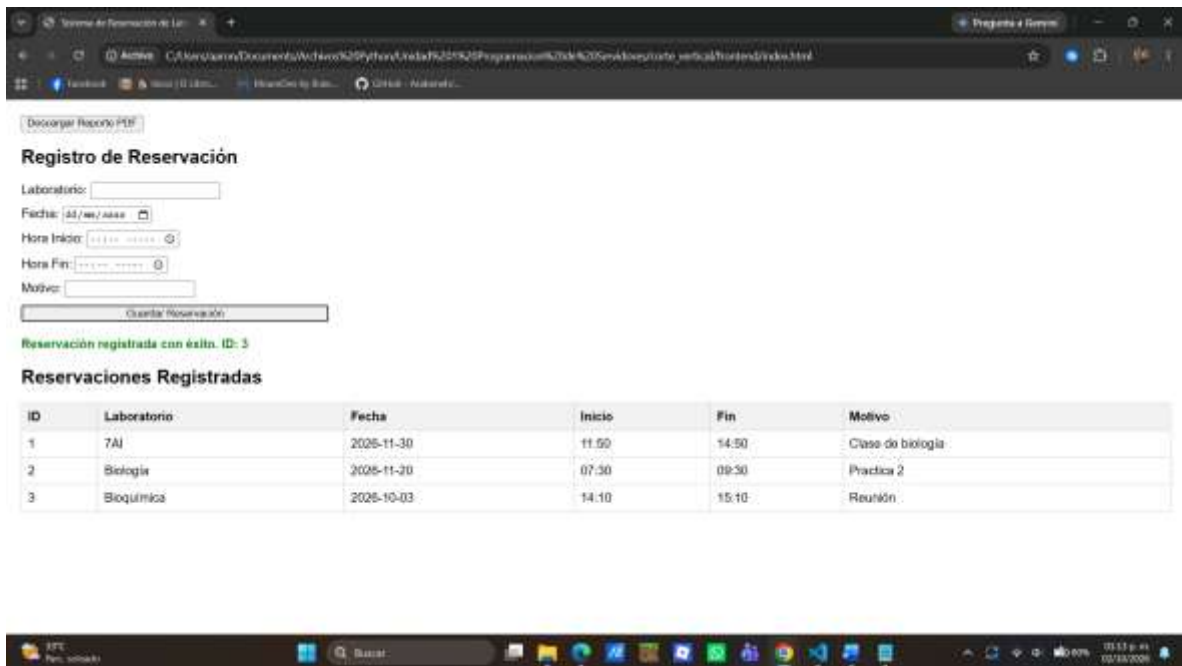


13.- Captura de Network.

```

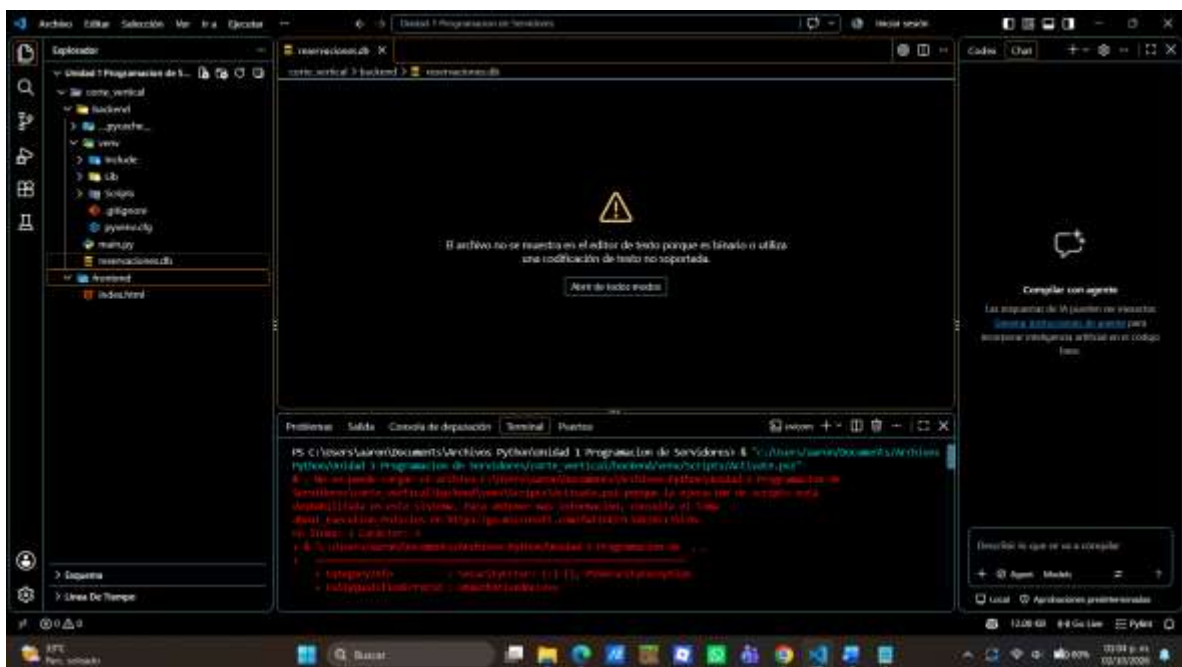
Problemas 14  Salida  Consola de depuración  Terminal  Puertos
uvicorn - backend

PS C:\Users\aaaron\Documents\Archivos Python\Unidad 1 Programacion de Servidores> cd corte_vertical
PS C:\Users\aaaron\Documents\Archivos Python\Unidad 1 Programacion de Servidores\corte_vertical> cd backend
PS C:\Users\aaaron\Documents\Archivos Python\Unidad 1 Programacion de Servidores\corte_vertical\backend> .\venv\Scripts
Activate.ps1
(venv) PS C:\Users\aaaron\Documents\Archivos Python\Unidad 1 Programacion de Servidores\corte_vertical\backend> uvicorn
main:app --reload
INFO: Will watch for changes in these directories: ['C:\Users\aaaron\Documents\Archivos Python\Unidad 1 Programacion de Servidores\corte_vertical\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [6292] using StatReload
INFO: Started server process [9700]
INFO: Waiting for application startup.
INFO: Application startup complete.
  
```

15.- Evidencia de error.

En este caso al uso del equipo, tener los scripts de forma nativa inhabilitado, no se puede correr durante visual studio code, entonces se buscó configurar la habilitación del mismo.



También, al principio hubo errores técnicos de extensiones, se buscó las soluciones pertinentes, y también por su parte que al tener la base de datos inhabilitada y se quiere hacer un registro, indica de forma visual que el backend está desactivado.

The screenshot shows a web browser window with the URL `C:\Programas\Documents\Windows\S25\Python\Unidad5\S25Programas\un5\7de6\S25Servidor\cliente_virtual\frontend/index.html`. The page has a dark theme and contains the following elements:

- A button labeled "Descargar Reporte PDF".
- A section titled "Registro de Reservación" with a form containing:
 - Laboratorio:
 - Fecha:
 - Hora Inicio:
 - Hora Fin:
 - Motivo:
 - A button labeled "Guardar Reservación".
- A red error message: "Error de conexión con el backend".
- A section titled "Reservaciones Registradas" with a table:

ID	Laboratorio	Fecha	Inicio	Fin	Motivo
----	-------------	-------	--------	-----	--------



This screenshot shows the same web application after a validation error. The form fields are now filled with different values:

- Laboratorio:
- Fecha:
- Hora Inicio:
- Hora Fin:
- Motivo:
- A button labeled "Guardar Reservación".

A red error message is displayed: "Error (422): La hora de inicio debe ser menor que la hora de fin". Below the form, the "Reservaciones Registradas" table is populated with two rows:

ID	Laboratorio	Fecha	Inicio	Fin	Motivo
1	7AJ	2026-11-30	11:50	14:50	Clase de biología
2	Biología	2026-11-20	07:30	09:30	Practica 2



En este caso por estar abarcando el horario de entrada la hora de salida, se marca el error 422.

16.- Explicación de la regla de negocio.

La regla de negocio previene el solapamiento de reservaciones en un mismo espacio y fecha.

Se evalúa en la consulta SQL mediante la condición:

`hora_inicio_existente < hora_fin_nueva and hora_fin_existente > hora_inicio_nueva`

Si la base de datos devuelve algún registro que cumpla esta condición para el mismo laboratorio y fecha, el backend interrumpe la ejecución y retorna un error 409 Conflict, impidiendo la duplicidad o cruce de horarios.

17.- Conclusiones.

La implementación del corte vertical permitió comprender la interacción real de un flujo de datos completo entre el frontend y el backend.

Por otra parte, se comprobó la importancia de validar las entradas tanto en el cliente como en el servidor y de manejar explícitamente los códigos de estado HTTP (201, 422, 409) para ofrecer una retroalimentación clara al usuario sobre el estado de sus peticiones.

Pero, sobre todo, los errores que se presentan en diversos equipos suelen ser específicos, dado que sean por incompatibilidad o por algún aspecto de seguridad es que se bloquean como lo son en el caso de los scripts.

Preguntas de defensa oral.

1.- ¿Dónde está el cliente?

En el navegador web (frontend), desarrollado con la interfaz en HTML, CSS y JavaScript que interactúa con el usuario.

2.- ¿Dónde está el servidor?

En el entorno backend donde se ejecuta la aplicación en FastAPI (Python), encargada de procesar la lógica de negocio y las peticiones.

3.- ¿Cuál es el endpoint?

Es la ruta de la API expuesta por el servidor para gestionar la reservación (por ejemplo, /api/v1/reservations o /reservaciones).

4.- ¿Qué método HTTP utiliza?

El método POST, ya que se está creando un nuevo recurso (una nueva reservación en el sistema).

5.- ¿Qué información envía el cliente y en qué formato?

Envía los datos de la reservación (como ID de usuario/laboratorio, fecha, hora inicio, hora fin) en formato JSON en el cuerpo (body) de la petición HTTP.

6.- ¿Dónde se validan los datos?

Se validan en dos puntos: una validación previa de formato en el cliente (HTML/JS) y la validación definitiva e ineludible en el backend (FastAPI con Pydantic).

7.- ¿Por qué no basta validar en HTML?

Porque la validación en HTML (frontend) es fácilmente evitable o manipulable desde la consola del navegador o mediante herramientas externas como Postman o cURL. La seguridad y consistencia del sistema dependen siempre de la validación en el servidor.

8.- ¿Qué regla de negocio implementaron?

La prevención de empalmes o conflictos de horario, asegurando que no se pueda reservar un mismo laboratorio en un intervalo de tiempo que ya esté ocupado por otra reservación previa.

9.- ¿Dónde se persiste la información?

En la base de datos SQLite conectada al backend.

10.- ¿Qué significa 201 Created?

Es el código de estado HTTP que indica que la petición tuvo éxito y que se creó correctamente un nuevo recurso (la reservación) en el servidor.

11.- ¿Cuándo utilizan 409?

Se utiliza (código 409 Conflict) cuando la petición no se puede procesar debido a un conflicto con el estado actual del recurso; en nuestro caso, cuando hay un traslape de horario con una reservación existente.

12.- ¿Puede Postman consumir la misma API?

Sí, totalmente. Al ser una API REST estándar que recibe JSON, cualquier cliente HTTP como Postman puede consumir los mismos endpoints sin problemas.

13.- ¿Puede Flutter consumirla?

Sí, exactamente de la misma manera que Postman o el sitio web, haciendo peticiones HTTP/JSON hacia la API backend.

14.- ¿Qué tendría que cambiar para sustituir HTML por Flutter?

Únicamente la interfaz gráfica en el cliente (capa de presentación). El backend en FastAPI, la base de datos SQLite y las reglas de negocio se quedan 100% idénticos, ya que están desacoplados del frontend.

15.- ¿Qué sucede si la BD deja de funcionar?

El servidor no puede guardar ni consultar datos, por lo que responderá con un error interno del servidor (código 500 Internal Server Error) informando que la persistencia falló.

16.- ¿Qué elemento representa el contrato API?

La especificación del endpoint (documentada habitualmente con OpenAPI/Swagger en FastAPI), que define la URL, método HTTP, parámetros, estructura del JSON de entrada/salida y códigos de estado devueltos.